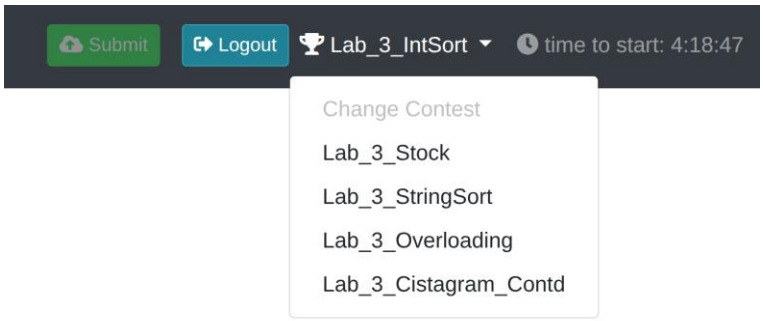
C++ Lab 3 Aug 19, 2026

There are **5 contests** on **Domjudge**.

Contest Name	**Deadline**
Lab3_ Stock	Friday, 21st Aug EOD
Lab3_IntSort	Friday, 21st Aug EOD
Lab3_StringSort	Friday, 21st Aug EOD
Lab3_Overloading	Friday, 21st Aug EOD
Lab3_Cistagram	Tuesday, 25st Aug EOD

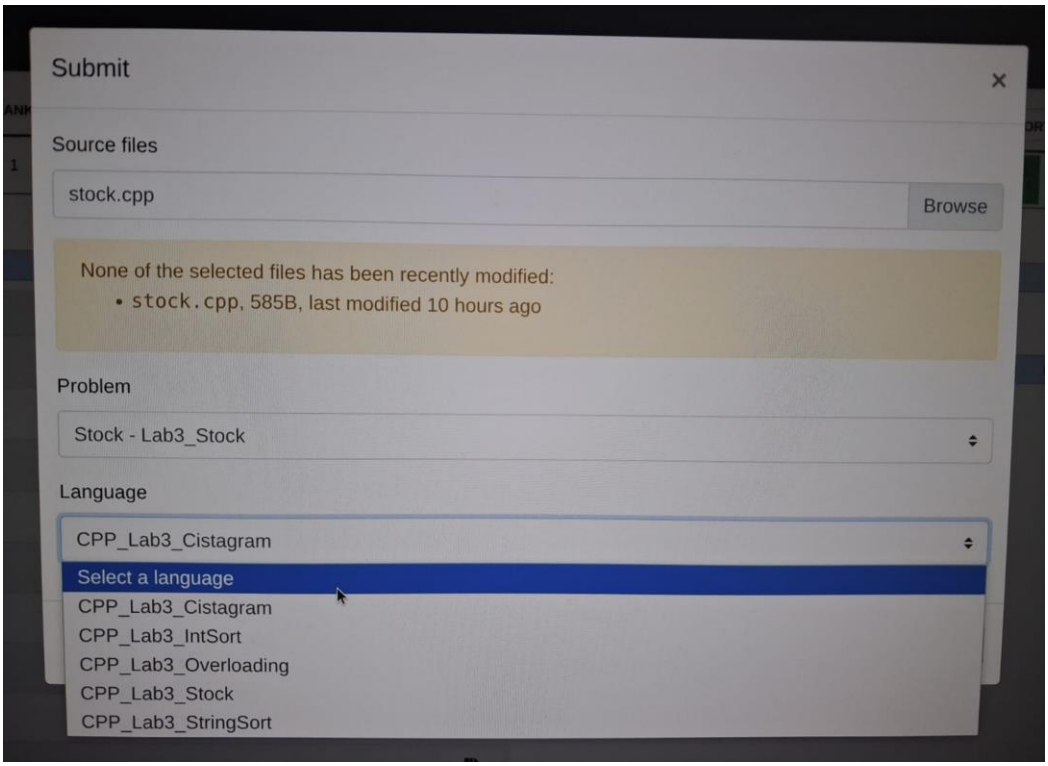
Steps to submit:

1) Select the contest (question) you want to submit to.



2) <u>IMPORTANT</u>: Select the corresponding Language from the dropdown.

There are specialized test scripts for each contest, hence use the correct script for the contest you are submitting to.



Some errors/ problems that you might face.

1) IMPORTANT: Language-Contest Mismatch

Diff output

```
Wrong Answer
Did not find required line:
Lab3_StringSort
```

This error arises when you submit your code for a contest but you select the wrong language.

2) Function Signature Mismatch Error

```
--- Missing Function Check ---
The compiler could not find the following required functions (check for typos):
has no member named 'findUserByName'; did you mean 'findUserByNam'?
-----------------------------
```

Such errors arise when the signature of the function (return type, name, parameters) does not match the expected signature.

3) Wrong Answer

Diff output

```
Wrong Answer
Did not find required line:
SCORE: 100
```

This means that your code complied successfully and passed few test cases but not all.

Domjudge is trying to find “SCORE:100” but since your code did not pass all cases, your score is not 100, hence it shows “Wrong Answer”.

Q1. store maintains the stock quantities of two products. Write a C++ program that:

1. Reads four integers:

   firstStock secondStock adjustment minimumStock

2. Creates a reference named stockRef for firstStock.

3. Adds adjustment to firstStock using stockRef.

4. Sets stockRef to minimumStock if it falls below minimumStock.

5. Calls orderStocks(firstStock, secondStock).

6. The function must swap the values only if firstStock > secondStock.

7. Prints the final stock values.

Implement the function using reference parameters:

void orderStocks(int &first, int &second);

INPUT FORMAT

firstStock secondStock adjustment minimumStock

OUTPUT FORMAT

firstStock secondStock

SAMPLE INPUT

70 40 10 0

SAMPLE OUTPUT

40 80

SAMPLE INPUT:

20 50 10 0

SAMPLE OUTPUT

30 50

Q2. Create a function called ***bubbleSort***, which takes an array of n integers, and returns the sorted array.

To implement it, first create one function void order(int& one, int& other), which puts one and other in ascending order – in the end, one should be the smaller one and other should be the larger one.

Write a C++ bubble-sort code that uses the order function to make bubble exchanges.

Now, create a class NumArray, which internally holds an array of 10 integers, and the bubbleSort private method (n=10 by default, so modify the method to use default parameter).
It should now have a public constructor to initialize the internal array with the parameter array data, and a public method sort(), which internally calls the bubbleSort method to sort it. Another constant public method show(), which should print the sorted array, **and also, return the array**. Remember, if it is return by reference (or pointer), the caller must not be able to modify the internal array from outside.

Create your own main to test this NumArray class.

Q3. Create a copy of the previous program, rename the class be to StringArray, to implement the same bubbleSort functionality for strings. You need to use #include<string> for this in your code, and know that for two string variables s1 and s2 can be compared using the standard > or < or >= or <= or == in the same manner that is applicable to integers. Now, keeping the class structure the same, modify all the internals to make it work for strings. Alphabetical order is sorted order here. Write your own main to test.

Q4. Implement the following overloaded min() functions
int min(int a, int b) ;
double min(double a, double b) ;
int min(int arr[], int len) ; // returns minimum of array.
Write your main that tests your three implementations with matching examples.

What happens if you call min(10, 100.2)?

Cistagram (cntd.)

Design Refactoring: Ctor, Dtor, Composition vs Aggregation

Q0. Identify the read-only methods/parameters in the class code (the classes which you are altering), and add “const” qualifier at proper places. Especially, for methods/functions that take string arguments, use const string& type of parameters appropriately, to prevent unnecessary copying of the string arguments.

Q1. Add a constructor for the User class, for creating a User object with a specified name. Remove the setUsername setter method of this class. Adjust the corresponding calls for object creating in the Cistagram program.

Q2.(a) Currently, a random photo gets created (it is mock for an external photo), and it’s get included in a post. When the post gets deleted, the photo also gets deleted (freeing memory). This is composition. Ideally, we want aggregation. So, in createPost, make a copy of the Photo object so that Post owns it. Make a global Photo array (max can hold 1000 photos) to hold the external photo objects (so that we do not loose access to these heap memories), which is explicitly freed just before the app exits. Write a clearExternalPhotos function to do that.

If total external photos exceed 1000 in numbers, createPost should fail. User can be given an error message like “no space left in the device”.

Q2.(b) clearUsers and clearPosts should now be captured in the destructors of the corresponding classes. That is, when the Cistagram app objects dies, all the Users die automatically, no need to call clearUsers explicitly. Similarly, when a User account gets deleted, all the Posts get automatically deleted (memory for the owned photos need to be freed). Incorporate this logic in the two classes, and remove the clear methods.